



IDX Trading Helper — PRD, Arsitektur & Starter Repo (PWA)



Disclaimer: Edukasi, bukan nasihat keuangan. Data dapat terlambat/berbeda.

Aplikasi ini **bukan auto-trading** dan **tidak mengeksekusi order**.

0. Ringkasan & Pilihan Tech Stack

Stack yang dipakai (sesuai rekomendasi, dengan alasan singkat):

Layer	Pilihan	Alasan
Framework	Next.js (App Router) + TypeScript	Frontend PWA + API proxy dalam satu repo; API key aman di server (Route Handlers).
PWA	next-pwa (Workbox di balik layar)	Konfigurasi runtime caching deklaratif, generate service worker otomatis.
UI	Tailwind CSS	Mobile-first, bundle kecil, dark mode bawaan via class.
Chart	lightweight-charts	~45KB gzip, paling ringan untuk candlestick/line trading chart.
Storage lokal	IndexedDB via Dexie	Offline-first untuk watchlist/plan/journal; query ergonomis.
Data fetching	TanStack Query	Cache, retry, stale-while-revalidate, dedup request market data.
Validasi	Zod	Validasi schema import/export dan input form.

1. PRD Ringkas

Tujuan

Membantu trader IDX melakukan analisis ringan dan menjaga **disiplin** (plan, risk, journal) dalam satu PWA mobile-first yang bisa dipakai offline untuk data pribadi.

In Scope (MVP)

- Watchlist IDX (CRUD) + quote ringkas + status koneksi.
- Detail saham: chart (candlestick/line) + SMA(20), EMA(20), RSI(14), timeframe 1D.
- Trading Plan dengan validasi logika (long).
- Risk Calculator (lot/lembar, konfigurabel).
- Journal + statistik (winrate, avg R, expectancy).
- Alert sederhana (price cross / RSI OB-OS) saat app terbuka, via Notifications API.
- PWA installable + offline app shell.
- Export/Import JSON (tanpa API key).

Out of Scope (MVP)

- Eksekusi order / koneksi broker.
- Realtime streaming/websocket & intraday tick.
- Background push notification (server push) & background sync kompleks.
- Multi-user / sinkronisasi cloud / akun.
- Backtesting engine, screener massal, fundamental data.
- Short selling logic (validasi plan fokus long dulu).

Asumsi & Batasan

- Provider gratis (Twelve Data / Alpha Vantage) memiliki **rate limit ketat** (mis. 8 req/menit, 800/hari pada free tier) dan **delay EOD/15 menit**. Karena itu market data di-cache agresif.
 - 1 lot IDX = 100 lembar (dibuat konfigurabel di Settings).
-

2. User Stories + Acceptance Criteria

US-1 Watchlist

Sebagai trader, saya ingin menambah/menghapus saham di watchlist agar bisa memantau cepat.

- ☐ Bisa add by kode (BBCA) + alias opsional; validasi kode tidak kosong & unik.
- ☐ Tiap item menampilkan last price, %change, last updated, dan badge status (online/stale/offline).
- ☐ Data watchlist tersimpan di IndexedDB dan tetap ada setelah app ditutup/offline.

US-2 Detail Saham

Sebagai trader, saya ingin melihat chart + indikator agar bisa analisis teknikal dasar.

- ☐ Chart daily tampil; toggle candlestick/line.
- ☐ Overlay SMA(20), EMA(20); panel RSI(14) di bawah.
- ☐ Saat data gagal/limit, tampil error state yang jelas + tombol retry; data lama (stale) tetap ditampilkan jika ada.

US-3 Trading Plan

Sebagai trader, saya ingin menyimpan plan agar disiplin entry/exit.

- ☐ Field: entryLow, entryHigh, stopLoss, tp1, tp2, riskPct, setup, thesis.
- ☐ Validasi long: $SL < EntryLow \leq EntryHigh < TP1 \leq TP2$; pelanggaran ditolak dengan pesan spesifik.
- ☐ Plan tersimpan per symbol di IndexedDB.

US-4 Risk Calculator

Sebagai trader, saya ingin tahu ukuran posisi sesuai risiko.

- ☐ Input: modal, risk%, entry, stopLoss, lotSize.
- ☐ Output: risk nominal = modal × risk%; risk/share = |entry - SL|; shares = floor(riskNominal / riskPerShare); lot = floor(shares / lotSize).
- ☐ Menangani SL = entry (bagi nol) dan input negatif dengan pesan error.

US-5 Journal

Sebagai trader, saya ingin mencatat trade & melihat statistik.

- ☐ Catat: tanggal, symbol, entry, exit, qty/lot, fees(opsional), alasan, emosi, hasil R.
- ☐ Statistik: jumlah trade, winrate, avg R, expectancy = $(winrate \times avgWinR) - (lossrate \times avgLossR)$.

US-6 Alert

Sebagai trader, saya ingin diingatkan saat harga/RSI menyentuh level.

- ☐ Rule: price crosses X (up/down); $RSI \geq 70$ (OB) / ≤ 30 (OS).
- ☐ Minta izin Notifications; jika ditolak, fallback ke in-app toast.
- ☐ Evaluasi alert berjalan saat app terbuka (polling sesuai cache).

US-7 PWA / Offline

- ☐ App bisa di-install dari browser mobile (manifest + SW).
- ☐ App shell cache-first; offline tetap bisa buka watchlist/journal/plan.

US-8 Export/Import

- ☐ Export seluruh IndexedDB ke satu file JSON (tanpa API key).
- ☐ Import memvalidasi schema (zod) sebelum menulis; tolak file invalid.

3. Data Model IndexedDB (Dexie Schema)

```
// src/lib/db/db.ts
import Dexie, { type Table } from "dexie"

export interface WatchItem {
  id?: number
  code: string // BBKA (uppercase, tanpa suffix)
  alias?: string
  createdAt: number
}

export interface TradingPlan {
```

```

    id?: number
    code: string
    direction: "long" // MVP long-only
    entryLow: number
    entryHigh: number
    stopLoss: number
    tp1: number
    tp2: number
    riskPct: number
    setup?: string
    thesis?: string
    updatedAt: number
  }

export interface JournalEntry {
  id?: number
  code: string
  date: string // ISO yyyy-mm-dd
  entry: number
  exit: number
  qtyLot: number
  fees?: number
  reason?: string
  emotion?: string
  rMultiple?: number // hasil dalam R
  createdAt: number
}

export interface AlertRule {
  id?: number
  code: string
  type: "price_cross" | "rsi"
  direction?: "above" | "below"
  price?: number
  rsiMode?: "overbought" | "oversold"
  active: boolean
  lastTriggeredAt?: number
}

```

```

export interface Setting {
  key: string          // e.g. "lotSize", "theme"
  value: unknown
}

export class IDXDatabase extends Dexie {
  watchlist!: Table<WatchItem, number>
  plans!: Table<TradingPlan, number>
  journal!: Table<JournalEntry, number>
  alerts!: Table<AlertRule, number>
  settings!: Table<Setting, string>

  constructor() {
    super("idx-trading-helper")
    this.version(1).stores({
      watchlist: "++id, &code, createdAt",
      plans: "++id, code, updatedAt",
      journal: "++id, code, date, createdAt",
      alerts: "++id, code, active",
      settings: "&key",
    })
  }
}

export const db = new IDXDatabase()

```

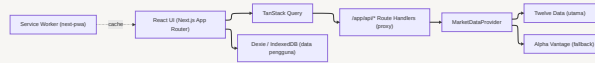
4. Arsitektur & Struktur Folder

```

flowchart LR
    UI["React UI (Next.js App Router)"] --> TQ["TanStack Query"]
    UI --> DX["Dexie / IndexedDB (data pengguna)"]
    TQ --> API["/app/api/* Route Handlers (proxy)"]
    API --> PROV["MarketDataProvider"]
    PROV --> TD["Twelve Data (utama)"]

```

PROV --> AV["Alpha Vantage (fallback)"]
 SW["Service Worker (next-pwa)"] -.cache.-> UI



Key: market API key **hanya** di server (Route Handler env), client hanya memanggil `/api/*`.

```

idx-trading-helper/
├─ public/
│  ├─ manifest.webmanifest
│  └─ icons/ (192, 512, maskable)
├─ src/
│  ├─ app/
│  │  ├─ layout.tsx
│  │  ├─ page.tsx # / Dashboard
│  │  ├─ providers.tsx # QueryClientProvider
│  │  ├─ symbol/[code]/page.tsx # detail
│  │  ├─ journal/page.tsx
│  │  ├─ settings/page.tsx
│  │  └─ api/
│  │     ├─ quote/route.ts
│  │     ├─ timeseries/daily/route.ts
│  │     └─ search/route.ts
│  └─ components/
│     ├─ WatchlistCard.tsx
│     ├─ PriceChart.tsx
│     ├─ RiskCalculator.tsx
│     ├─ TradingPlanForm.tsx
│     └─ ConnectionBadge.tsx
├─ lib/
│  ├─ db/db.ts
│  ├─ market/provider.ts # interface
│  ├─ market/twelveData.ts
│  ├─ market/alphaVantage.ts
│  ├─ market/index.ts # composite + fallback
│  ├─ market/symbol.ts # normalisasi
│  └─ indicators.ts # SMA/EMA/RSI
  
```

```

| | └─ risk.ts
| | └─ plan.ts # validasi plan
| | └─ stats.ts # journal stats
| | └─ backup.ts # export/import + zod
| └─ hooks/ (useQuote, useTimeSeries, useWatchlist...)
└─ next.config.mjs
└─ tailwind.config.ts
└─ .env.local.example
└─ package.json

```

5. Wireframe Deskriptif per Halaman

/ (Dashboard) — header (judul + ConnectionBadge), input quick-add (kode + alias) full-width, list WatchlistCard (kode, alias, harga besar, %change berwarna, last updated kecil), section "Alert aktif" di bawah. Sticky bottom nav: Dashboard / Journal / Settings.

/symbol/[code] — judul kode + harga + %change; toggle candlestick/line + toggle SMA/EMA; chart utama; panel RSI di bawah; accordion "Trading Plan" (form + validasi inline); tombol primer "Add journal entry"; RiskCalculator collapsible.

/journal — kartu statistik atas (trades, winrate, avg R, expectancy) dalam grid 2 kolom; daftar entry (tanggal, symbol, R berwarna); FAB tambah entry.

/settings — toggle dark mode; izin notifikasi (status + tombol minta izin); lotSize input; provider status (Twelve Data / Alpha Vantage: ok/limit/error); tombol Export & Import.

6. Milestones Implementasi

1. **M1 — Skeleton & PWA:** Next.js + TS + Tailwind + next-pwa + manifest/icons + layout + bottom nav + dark mode. *Deliverable: app installable, app shell offline.*
2. **M2 — Data layer & proxy:** Dexie schema, MarketDataProvider + Twelve Data + Alpha Vantage fallback + symbol normalize + unit test, route `/api/quote`, `/api/timeseries/daily`, `/api/search`, TanStack Query hooks.
3. **M3 — Fitur inti:** Watchlist CRUD UI + quote, detail page + chart + SMA/EMA/RSI, Trading Plan + validasi, Risk Calculator.

4. **M4 — Journal, Alert, Backup, QA:** Journal + statistik, alert (price/RSI) + Notifications API, export/import zod, error states, aksesibilitas, QA mobile.
-

7. Kode Awal (Runnable)

7.1 Setup

```
npx create-next-app@latest idx-trading-helper --typescript
--tailwind --app --eslint --src-dir --import-alias "@/*"
cd idx-trading-helper
npm i dexie @tanstack/react-query lightweight-charts zod
npm i -D next-pwa vitest @vitest/coverage-v8
```

`.env.local.example`

```
TWELVE_DATA_API_KEY=your_twelve_data_key
ALPHA_VANTAGE_API_KEY=your_alpha_vantage_key
# MIC IDX untuk Twelve Data
NEXT_PUBLIC_DEFAULT_LOT_SIZE=100
```

7.2 next.config.mjs (next-pwa + caching)

```
import withPWAInit from "next-pwa"

const withPWA = withPWAInit({
  dest: "public",
  disable: process.env.NODE_ENV === "development",
  runtimeCaching: [
    {
      // App shell: cache-first
      urlPattern: ({ request }) =>
        ["document", "script", "style", "image", "font"].includes(request.destination),
      handler: "CacheFirst",
      options: { cacheName: "app-shell", expiration: {
        maxEntries: 100 } },
    },
    {
```

```

        // Market data API: network-first + stale fallback
ack
        urlPattern: ({ url }) => url.pathname.startsWith(
h("/api/"),
        handler: "NetworkFirst",
        options: {
            cacheName: "market-api",
            networkTimeoutSeconds: 5,
            expiration: { maxEntries: 200, maxAgeSeconds: 60 * 60 * 24 },
        },
    ],
})

/** @type {import('next').NextConfig} */
const nextConfig = { reactStrictMode: true }
export default withPWA(nextConfig)

```

public/manifest.webmanifest

```

{
  "name": "IDX Trading Helper",
  "short_name": "IDX Helper",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0a0a0a",
  "theme_color": "#0a0a0a",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192",
"type": "image/png" },
    { "src": "/icons/icon-512.png", "sizes": "512x512",
"type": "image/png" },
    { "src": "/icons/maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ]
}

```

7.3 Symbol Normalisasi (riset format) + Unit Test

Hasil riset format simbol IDX per provider:

Sumber	Format BBCA	Catatan
Input user	BBCA	kode 4 huruf, uppercase
Twelve Data	symbol=BBCA + mic_code=XIDX	MIC IDX = XIDX (atau exchange=XIDX / country=Indonesia)
Alpha Vantage	BBCA.JKT	suffix Jakarta = .JKT
Yahoo/Google (ref)	BBCA.JK / BBCA:IDX	bukan dipakai, hanya rujukan

```
// src/lib/market/symbol.ts
export type ProviderId = "twelvedata" | "alphavantage"

export const IDX_MIC = "XIDX"

/** Bersihkan input user ke kode dasar IDX: uppercase, hapus suffix umum & spasi. */
export function normalizeUserCode(raw: string): string {
  if (!raw) throw new Error("Kode saham kosong")
  const code = raw
    .trim()
    .toUpperCase()
    .replace(/\s+/g, "")
    .replace(/[:.](JK|JKT|IDX|XIDX)$/i, "") // buang suffix jika user terlanjur menuliskannya
  if (!/^[A-Z]{2,5}$/.test(code)) throw new Error(`Kode IDX tidak valid: ${raw}`)
  return code
}

/** Petakan kode dasar ke format simbol provider. */
export function toProviderSymbol(code: string, provider: ProviderId): string {
  const base = normalizeUserCode(code)
  switch (provider) {
    case "twelvedata":
      return base // exchange/mic dikirim terpisah (m
```

```

ic_code=XIDX)
    case "alphavantage":
        return `${base}.JKT`
    }
}

```

```

// src/lib/market/symbol.test.ts
import { describe, it, expect } from "vitest"
import { normalizeUserCode, toProviderSymbol } from "../symbol"

describe("normalizeUserCode", () => {
  it("uppercases & trims", () => {
    expect(normalizeUserCode(" bbca ")).toBe("BBCA")
  })
  it("strips suffixes", () => {
    expect(normalizeUserCode("BBCA.JK")).toBe("BBCA")
    expect(normalizeUserCode("tlkm.jkt")).toBe("TLKM")
    expect(normalizeUserCode("ASII:IDX")).toBe("ASII")
  })
  it("rejects invalid", () => {
    expect(() => normalizeUserCode("")).toThrow()
    expect(() => normalizeUserCode("12!")).toThrow()
  })
})

describe("toProviderSymbol", () => {
  it("twelvedata = base", () => {
    expect(toProviderSymbol("BBCA", "twelvedata")).toBe("BBCA")
  })
  it("alphavantage = .JKT", () => {
    expect(toProviderSymbol("bbca", "alphavantage")).toBe("BBCA.JKT")
  })
})

```

7.4 MarketDataProvider (interface + impl + fallback)

```
// src/lib/market/provider.ts
export interface Quote {
  symbol: string
  price: number
  change: number
  changePercent: number
  timestamp: number
}

export interface Candle {
  time: string // yyyy-mm-dd
  open: number; high: number; low: number; close: number;
  volume: number
}

export interface SymbolResult { code: string; name: string
}

export interface MarketDataProvider {
  id: "twelvedata" | "alphavantage"
  searchSymbol(query: string): Promise<SymbolResult[]>
  getQuote(code: string): Promise<Quote>
  getTimeSeriesDaily(code: string, outputsize?: number):
  Promise<Candle[]>
}

export class RateLimitError extends Error {}
export class InvalidSymbolError extends Error {}
export class NoDataError extends Error {}
```

```
// src/lib/market/twelveData.ts
import { IDX_MIC, toProviderSymbol } from "../symbol"
import { type MarketDataProvider, type Quote, type Candle,
  type SymbolResult, RateLimitError, InvalidSymbolError, NoDataError } from "../provider"

const BASE = "https://api.twelvedata.com"
```

```

function key() {
  const k = process.env.TWELVE_DATA_API_KEY
  if (!k) throw new Error("TWELVE_DATA_API_KEY missing")
  return k
}

function assertOk(json: any) {
  if (json?.status === "error") {
    const msg = String(json.message ?? "")
    if (/run out of API credits|limit/i.test(msg)) throw new RateLimitError(msg)
    if (/not found|symbol/i.test(msg)) throw new InvalidSymbolError(msg)
    throw new NoDataError(msg)
  }
}

export const twelveData: MarketDataProvider = {
  id: "twelvedata",
  async searchSymbol(query) {
    const r = await fetch(`${BASE}/symbol_search?symbol=${encodeURIComponent(query)}&apikey=${key()}`)
    const j = await r.json(); assertOk(j)
    return (j.data ?? []).filter((d: any) => d.mic_code === IDX_MIC || d.country === "Indonesia").map((d: any): SymbolResult => ({ code: d.symbol, name: d.instrument_name }))
  },
  async getQuote(code) {
    const sym = toProviderSymbol(code, "twelvedata")
    const r = await fetch(`${BASE}/quote?symbol=${sym}&mic_code=${IDX_MIC}&apikey=${key()}`)
    const j = await r.json(); assertOk(j)
    if (!j.close) throw new NoDataError("empty quote")
    return {
      symbol: code,
      price: Number(j.close),
      change: Number(j.change),
    }
  }
}

```

```

        changePercent: Number(j.percent_change),
        timestamp: Number(j.timestamp) * 1000 || Date.now(),
    },
    },
    async getTimeSeriesDaily(code, outputsize = 200) {
        const sym = toProviderSymbol(code, "twelvedata")
        const r = await fetch(`${BASE}/time_series?symbol=${sym}&mic_code=${IDX_MIC}&interval=1day&outputsize=${outputsize}&apikey=${key()}`)
        const j = await r.json(); assertOk(j)
        if (!j.values) throw new NoDataError("empty series")
        return (j.values as any[]).reverse().map((v): Candle => ({
            time: v.datetime, open: +v.open, high: +v.high,
            low: +v.low, close: +v.close, volume: +v.volume,
        })))
    },
}

```

```

// src/lib/market/alphaVantage.ts
import { toProviderSymbol } from "./symbol"
import { type MarketDataProvider, type Candle, type Quote, type SymbolResult, RateLimitError, NoDataError } from "./provider"

const BASE = "https://www.alphavantage.co/query"
function key() {
    const k = process.env.ALPHA_VANTAGE_API_KEY
    if (!k) throw new Error("ALPHA_VANTAGE_API_KEY missing")
    return k
}
function guard(j: any) {
    if (j?.Note || j?.Information) throw new RateLimitError(j.Note ?? j.Information)
    if (j?.["Error Message"]) throw new NoDataError(j["Error Message"])
}

```

```

    r Message"]])
  }

export const alphaVantage: MarketDataProvider = {
  id: "alphavantage",
  async searchSymbol(query) {
    const r = await fetch(`${BASE}?function=SYMBOL_SEAR
CH&keywords=${encodeURIComponent(query)}&apikey=${key()}`)
    const j = await r.json(); guard(j)
    return (j.bestMatches ?? []).
      .filter((m: any) => m["4. region"] === "Jakarta")
      .map((m: any): SymbolResult => ({ code: String
(m["1. symbol"]).replace(/\.JKT$/i, ""), name: m["2. name"]
})))
  },
  async getQuote(code) {
    const sym = toProviderSymbol(code, "alphavantage")
    const r = await fetch(`${BASE}?function=GLOBAL_QUOT
E&symbol=${sym}&apikey=${key()}`)
    const j = await r.json(); guard(j)
    const q = j["Global Quote"]
    if (!q || !q["05. price"]) throw new NoDataError("e
mpty quote")
    return {
      symbol: code,
      price: +q["05. price"],
      change: +q["09. change"],
      changePercent: parseFloat(q["10. change percen
t"]),
      timestamp: Date.now(),
    }
  },
  async getTimeSeriesDaily(code, outputsize = 200) {
    const sym = toProviderSymbol(code, "alphavantage")
    const size = outputsize > 100 ? "full" : "compact"
    const r = await fetch(`${BASE}?function=TIME_SERIES
_DAILY&symbol=${sym}&outputsize=${size}&apikey=${key()}`)

```



```

const j = await r.json(); guard(j)
const series = j["Time Series (Daily)"]
if (!series) throw new NoDataError("empty series")
return Object.entries(series)
    .map(([time, v]: [string, any]): Candle => ({
        time, open: +v["1. open"], high: +v["2. high"],
        low: +v["3. low"], close: +v["4. close"], volume: +v["5. volume"],
    }))
    .sort((a, b) => a.time.localeCompare(b.time))
    .slice(-outputsize)
},
}

```

```

// src/lib/market/index.ts – komposit dengan fallback
import { twelveData } from "./twelveData"
import { alphaVantage } from "./alphaVantage"
import { RateLimitError, type MarketDataProvider } from "./provider"

const chain: MarketDataProvider[] = [twelveData, alphaVantage]

async function withFallback<T>(fn: (p: MarketDataProvider) => Promise<T>): Promise<{ data: T; provider: string }> {
    let lastErr: unknown
    for (const p of chain) {
        try { return { data: await fn(p), provider: p.id } }
        catch (e) {
            lastErr = e
            // hanya fallback saat limit/error sementara
            if (!(e instanceof RateLimitError)) throw e
        }
    }
    throw lastErr
}

```

```
export const market = {
  searchSymbol: (q: string) => withFallback((p) => p.searchSymbol(q)),
  getQuote: (code: string) => withFallback((p) => p.getQuote(code)),
  getTimeSeriesDaily: (code: string, n?: number) => withFallback((p) => p.getTimeSeriesDaily(code, n)),
}
```

7.5 API Proxy Routes

```
// src/app/api/quote/route.ts
import { NextResponse } from "next/server"
import { market } from "@lib/market"
import { RateLimitError, InvalidSymbolError, NoDataError }
from "@lib/market/provider"

export const runtime = "nodejs"

export async function GET(req: Request) {
  const symbol = new URL(req.url).searchParams.get("symbol")
  if (!symbol) return NextResponse.json({ error: "symbol required" }, { status: 400 })
  try {
    const { data, provider } = await market.getQuote(symbol)
    return NextResponse.json({ ...data, provider }, {
      headers: { "Cache-Control": "s-maxage=60, stale-while-revalidate=300" },
    })
  } catch (e) {
    if (e instanceof RateLimitError) return NextResponse.json({ error: "rate_limit" }, { status: 429 })
    if (e instanceof InvalidSymbolError) return NextResponse.json({ error: "invalid_symbol" }, { status: 404 })
    if (e instanceof NoDataError) return NextResponse.json({ error: "no_data" }, { status: 204 })
  }
}
```

```

        return NextResponse.json({ error: "upstream" }, { status: 502 })
    }
}

```

```

// src/app/api/timeseries/daily/route.ts
import { NextResponse } from "next/server"
import { market } from "@lib/market"
import { RateLimitError, InvalidSymbolError, NoDataError } from "@lib/market/provider"

export const runtime = "nodejs"

export async function GET(req: Request) {
    const sp = new URL(req.url).searchParams
    const symbol = sp.get("symbol")
    const outputsize = Number(sp.get("outputsize") ?? 200)
    if (!symbol) return NextResponse.json({ error: "symbol required" }, { status: 400 })
    try {
        const { data, provider } = await market.getTimeSeriesDaily(symbol, outputsize)
        return NextResponse.json({ candles: data, provider }, {
            headers: { "Cache-Control": "s-maxage=3600, stale-while-revalidate=86400" },
        })
    } catch (e) {
        if (e instanceof RateLimitError) return NextResponse.json({ error: "rate_limit" }, { status: 429 })
        if (e instanceof InvalidSymbolError) return NextResponse.json({ error: "invalid_symbol" }, { status: 404 })
        if (e instanceof NoDataError) return NextResponse.json({ error: "no_data" }, { status: 204 })
        return NextResponse.json({ error: "upstream" }, { status: 502 })
    }
}

```

```

    }
  }
}

```

7.6 Indikator (SMA/EMA/RSI) + Test

```

// src/lib/indicators.ts
export function sma(values: number[], period: number): (number | null)[] {
  const out: (number | null)[] = []
  let sum = 0
  for (let i = 0; i < values.length; i++) {
    sum += values[i]
    if (i >= period) sum -= values[i - period]
    out.push(i >= period - 1 ? sum / period : null)
  }
  return out
}

export function ema(values: number[], period: number): (number | null)[] {
  const out: (number | null)[] = []
  const k = 2 / (period + 1)
  let prev: number | null = null
  for (let i = 0; i < values.length; i++) {
    if (i < period - 1) { out.push(null); continue }
    if (prev === null) {
      prev = values.slice(0, period).reduce((a, b) =>
a + b, 0) / period
    } else {
      prev = values[i] * k + prev * (1 - k)
    }
    out.push(prev)
  }
  return out
}

export function rsi(values: number[], period = 14): (number | null)[] {

```

```

    const out: (number | null)[] = Array(values.length).fill(
      null
    )
    if (values.length <= period) return out
    let gain = 0, loss = 0
    for (let i = 1; i <= period; i++) {
      const d = values[i] - values[i - 1]
      if (d >= 0) gain += d; else loss -= d
    }
    let avgGain = gain / period, avgLoss = loss / period
    out[period] = avgLoss === 0 ? 100 : 100 - 100 / (1 + avgGain / avgLoss)
    for (let i = period + 1; i < values.length; i++) {
      const d = values[i] - values[i - 1]
      const g = d > 0 ? d : 0, l = d < 0 ? -d : 0
      avgGain = (avgGain * (period - 1) + g) / period
      avgLoss = (avgLoss * (period - 1) + l) / period
      out[i] = avgLoss === 0 ? 100 : 100 - 100 / (1 + avgGain / avgLoss)
    }
    return out
  }
}

```

```

// src/lib/indicators.test.ts
import { describe, it, expect } from "vitest"
import { sma, ema, rsi } from "./indicators"

describe("sma", () => {
  it("computes window avg", () => {
    expect(sma([1, 2, 3, 4], 2)).toEqual([null, 1.5, 2.5, 3.5])
  })
})

describe("rsi", () => {
  it("returns 100 when only gains", () => {
    const r = rsi([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], 14)
    expect(r[14]).toBe(100)
  })
})

```

```

    })
  })
}

```

7.7 Risk Calculator + Validasi Plan + Stats

```

// src/lib/risk.ts
export interface RiskInput { capital: number; riskPct: number; entry: number; stopLoss: number; lotSize?: number }
export interface RiskOutput { riskNominal: number; riskPerShare: number; shares: number; lots: number }

export function calcPosition(i: RiskInput): RiskOutput {
  const lotSize = i.lotSize ?? 100
  if (i.capital <= 0 || i.riskPct <= 0) throw new Error("Modal & risk% harus > 0")
  const riskPerShare = Math.abs(i.entry - i.stopLoss)
  if (riskPerShare === 0) throw new Error("Entry tidak boleh sama dengan Stop Loss")
  const riskNominal = i.capital * (i.riskPct / 100)
  const shares = Math.floor(riskNominal / riskPerShare)
  const lots = Math.floor(shares / lotSize)
  return { riskNominal, riskPerShare, shares, lots }
}

```

```

// src/lib/plan.ts
export interface PlanInput { entryLow: number; entryHigh: number; stopLoss: number; tp1: number; tp2: number }
// Long: SL < EntryLow <= EntryHigh < TP1 <= TP2
export function validateLongPlan(p: PlanInput): { ok: boolean; error?: string } {
  if (!(p.stopLoss < p.entryLow)) return { ok: false, error: "Stop Loss harus < Entry Low" }
  if (!(p.entryLow <= p.entryHigh)) return { ok: false, error: "Entry Low harus <= Entry High" }
  if (!(p.entryHigh < p.tp1)) return { ok: false, error: "Entry High harus < TP1" }
  if (!(p.tp1 <= p.tp2)) return { ok: false, error: "TP1

```

```

harus <= TP2" }
    return { ok: true }
}

```

```

// src/lib/stats.ts
import type { JournalEntry } from "../db/db"
export function journalStats(entries: JournalEntry[]) {
    const rs = entries.map((e) => e.rMultiple ?? 0)
    const n = rs.length
    if (!n) return { trades: 0, winrate: 0, avgR: 0, expectancy: 0 }
    const wins = rs.filter((r) => r > 0)
    const losses = rs.filter((r) => r < 0)
    const winrate = wins.length / n
    const avgWin = wins.length ? wins.reduce((a, b) => a + b, 0) / wins.length : 0
    const avgLoss = losses.length ? Math.abs(losses.reduce((a, b) => a + b, 0) / losses.length) : 0
    const expectancy = winrate * avgWin - (1 - winrate) * avgLoss
    const avgR = rs.reduce((a, b) => a + b, 0) / n
    return { trades: n, winrate, avgR, expectancy }
}

```

7.8 Providers + TanStack Query + Hooks

```

// src/app/providers.tsx
"use client"
import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
import { useState } from "react"
export default function Providers({ children }: { children: React.ReactNode }) {
    const [client] = useState(() => new QueryClient({
        defaultOptions: { queries: { staleTime: 60_000, retry: 1, refetchOnWindowFocus: false } },
    }))
}

```

```

    return <QueryClientProvider client={client}>{children}
  </QueryClientProvider>
}

```

```

// src/hooks/useMarket.ts
"use client"
import { useQuery } from "@tanstack/react-query"
export function useQuote(code: string) {
  return useQuery({
    queryKey: ["quote", code],
    queryFn: async () => {
      const r = await fetch(`/api/quote?symbol=${code}`)
      if (r.status === 429) throw new Error("rate_limit")
      if (!r.ok) throw new Error("quote_failed")
      return r.json()
    },
    enabled: !!code,
    refetchInterval: 60_000,
  })
}
export function useDaily(code: string) {
  return useQuery({
    queryKey: ["daily", code],
    queryFn: async () => {
      const r = await fetch(`/api/timeseries/daily?symbol=${code}`)
      if (!r.ok) throw new Error("series_failed")
      return r.json() as Promise<{ candles: any[]; provider: string }>
    },
    enabled: !!code,
    staleTime: 60 * 60 * 1000,
  })
}

```



```
// src/hooks/useWatchlist.ts – Dexie live
"use client"
import { useLiveQuery } from "dexie-react-hooks" // npm i dexie-react-hooks
import { db, type WatchItem } from "@lib/db/db"
import { normalizeUserCode } from "@lib/market/symbol"

export function useWatchlist() {
  const items = useLiveQuery(() => db.watchlist.orderBy(
    "createdAt").toArray(), [], [] as WatchItem[])
  async function add(code: string, alias?: string) {
    const c = normalizeUserCode(code)
    const exists = await db.watchlist.where("code").equals(c).count()
    if (exists) throw new Error("Sudah ada di watchlist")
    await db.watchlist.add({ code: c, alias, createdAt: Date.now() })
  }
  const remove = (id: number) => db.watchlist.delete(id)
  return { items, add, remove }
}
```

7.9 Watchlist UI (Dashboard) + Detail + Chart

```
// src/app/page.tsx
"use client"
import { useState } from "react"
import Link from "next/link"
import { useWatchlist } from "@hooks/useWatchlist"
import { useQuote } from "@hooks/useMarket"

function Row({ code, alias, onRemove }: { code: string; alias?: string; onRemove: () => void }) {
  const { data, isError, isLoading } = useQuote(code)
  const up = (data?.changePercent ?? 0) >= 0
  return (
```

```

    <div className="flex items-center justify-between rounded-xl border p-3 dark:border-neutral-800">
      <Link href={` /symbol/${code}`} className="flex-1">
        <div className="font-semibold">{code} {alias} && <span className="text-xs opacity-60">{alias}</span></div>
        <div className="text-xs opacity-60">
          {isLoading ? "memuat..." : isError ? "gagal/limit" : `upd ${new Date(data.timestamp).toLocaleTimeString()}`}
        </div>
        </Link>
        <div className={`text-right ${up ? "text-green-500" : "text-red-500"}`}>
          <div className="font-bold">{data?.price ?? "-"}</div>
          <div className="text-xs">{data ? `${data.changePercent?.toFixed(2)}%` : ""}</div>
        </div>
        <button aria-label="hapus" onClick={onRemove} className="ml-3 opacity-50">x</button>
      </div>
    )
  }

export default function Dashboard() {
  const { items, add, remove } = useWatchlist()
  const [code, setCode] = useState(""); const [alias, setAlias] = useState(""); const [err, setErr] = useState("")
  async function submit(e: React.FormEvent) {
    e.preventDefault(); setErr("")
    try { await add(code, alias || undefined); setCode(""); setAlias("") }
    catch (x: any) { setErr(x.message) }
  }
  return (
    <main className="mx-auto max-w-md space-y-3 p-4">

```

```

        <h1 className="text-xl font-bold">IDX Trading H
elper</h1>
        <form onSubmit={submit} className="flex gap-2">
            <input value={code} onChange={(e) => setCod
e(e.target.value)} placeholder="BBCA" className="flex-1 rou
nded-lg border p-2 dark:bg-neutral-900" />
            <input value={alias} onChange={(e) => setAl
ias(e.target.value)} placeholder="alias" className="w-24 ro
unded-lg border p-2 dark:bg-neutral-900" />
            <button className="rounded-lg bg-blue-600 p
x-4 text-white">+</button>
        </form>
        {err && <p className="text-sm text-red-500">{er
r}</p>}}
        <div className="space-y-2">
            {items?.map((it) => <Row key={it.id} code=
{it.code} alias={it.alias} onRemove={() => remove(it.id!)}
/>)}
        </div>
    </main>
)
}

```

```

// src/components/PriceChart.tsx
"use client"
import { useEffect, useRef } from "react"
import { createChart, type IChartApi } from "lightweight-ch
arts"
import { sma, ema } from "@lib/indicators"

export function PriceChart({ candles }: { candles: { time:
string; open: number; high: number; low: number; close: num
ber }[] }) {
    const ref = useRef<HTMLDivElement>(null)
    const chart = useRef<IChartApi | null>(null)
    useEffect(() => {
        if (!ref.current) return
        chart.current = createChart(ref.current, { height:

```

```

320, layout: { background: { color: "transparent" }, textCo
lor: "#888" }, grid: { vertLines: { visible: false } } })
    const candle = chart.current.addCandlestickSeries()
    candle.setData(candles as any)
    const closes = candles.map((c) => c.close)
    const sma20 = chart.current.addLineSeries({ lineWid
th: 1 })
    sma20.setData(sma(closes, 20).map((v, i) => v == nu
ll ? null : { time: candles[i].time, value: v }).filter(Boo
lean) as any)
    const ema20 = chart.current.addLineSeries({ lineWid
th: 1, color: "#f59e0b" })
    ema20.setData(ema(closes, 20).map((v, i) => v == nu
ll ? null : { time: candles[i].time, value: v }).filter(Boo
lean) as any)
    const ro = new ResizeObserver(() => chart.current?.
applyOptions({ width: ref.current!.clientWidth }))
    ro.observe(ref.current)
    return () => { ro.disconnect(); chart.current?.remo
ve() }
  }, [candles])
  return <div ref={ref} className="w-full" />
}

```

7.10 Export / Import (Zod)

```

// src/lib/backup.ts
import { z } from "zod"
import { db } from "../db/db"

const backupSchema = z.object({
  version: z.literal(1),
  watchlist: z.array(z.object({ code: z.string(), alias:
z.string().optional(), createdAt: z.number() })),
  plans: z.array(z.any()),
  journal: z.array(z.any()),
  alerts: z.array(z.any()),
  settings: z.array(z.object({ key: z.string(), value: z.

```

```

    unknown() })),
  })

export async function exportData(): Promise<string> {
  // CATATAN: tidak ada API key di IndexedDB – keys hanya
  di env server.
  const [watchlist, plans, journal, alerts, settings] = a
  wait Promise.all([
    db.watchlist.toArray(), db.plans.toArray(), db.jour
    nal.toArray(), db.alerts.toArray(), db.settings.toArray(),
    ])
  return JSON.stringify({ version: 1, watchlist, plans, j
    urnal, alerts, settings }, null, 2)
}

export async function importData(json: string) {
  const parsed = backupSchema.parse(JSON.parse(json)) //
  throw jika invalid
  await db.transaction("rw", db.watchlist, db.plans, db.j
    urnal, db.alerts, db.settings, async () => {
    await Promise.all([db.watchlist.clear(), db.plans.c
    lear(), db.journal.clear(), db.alerts.clear(), db.settings.
    clear()])
    await db.watchlist.bulkAdd(parsed.watchlist as any)
    await db.plans.bulkAdd(parsed.plans as any)
    await db.journal.bulkAdd(parsed.journal as any)
    await db.alerts.bulkAdd(parsed.alerts as any)
    await db.settings.bulkAdd(parsed.settings as any)
  })
}

```

7.11 Alert (Notifications API, saat app terbuka)

```

// src/lib/alerts.ts
import { db } from "../db/db"

export async function ensureNotifyPermission(): Promise<boo
  lean> {

```

```

    if (!("Notification" in window)) return false
    if (Notification.permission === "granted") return true
    if (Notification.permission === "denied") return false
    return (await Notification.requestPermission()) === "granted"
  }

export async function evaluateAlerts(code: string, price: number, rsiNow: number | null) {
  const rules = await db.alerts.where("code").equals(code).and((r) => r.active).toArray()
  for (const r of rules) {
    let hit = false
    if (r.type === "price_cross" && r.price !== null)
      hit = r.direction === "above" ? price >= r.price : price <= r.price
    if (r.type === "rsi" && rsiNow !== null)
      hit = r.rsiMode === "overbought" ? rsiNow >= 70 : rsiNow <= 30
    if (hit) {
      const ok = await ensureNotifyPermission()
      const msg = `${code}: ${r.type === "rsi" ? `RSI ${r.rsiMode}` : `harga ${r.direction} ${r.price}`}`
      if (ok) new Notification("IDX Alert", { body: msg })
      else window.dispatchEvent(new CustomEvent("idx-toast", { detail: msg })) // fallback in-app
      await db.alerts.update(r.id!, { lastTriggeredAt: Date.now() })
    }
  }
}

```

7.12 app/layout.tsx (Providers, metadata, bottom nav)

```

// src/app/layout.tsx
import type { Metadata, Viewport } from "next"

```

```

import Link from "next/link"
import Providers from "../providers"
import "../globals.css"

export const metadata: Metadata = {
  title: "IDX Trading Helper",
  description: "Analisis & disiplin trading IDX. Edukasi, bukan nasihat keuangan.",
  manifest: "/manifest.webmanifest",
  appleWebApp: { capable: true, statusBarStyle: "black-translucent", title: "IDX Helper" },
}

export const viewport: Viewport = {
  themeColor: "#0a0a0a",
  width: "device-width",
  initialScale: 1,
  viewportFit: "cover",
}

function BottomNav() {
  const items = [
    { href: "/", label: "Watchlist", icon: "📊" },
    { href: "/journal", label: "Journal", icon: "📅" },
    { href: "/settings", label: "Settings", icon: "⚙️" },
  ],
  return (
    <nav className="fixed inset-x-0 bottom-0 z-10 flex border-t bg-white/90 backdrop-blur dark:border-neutral-800 dark:bg-neutral-950/90">
      {items.map((it) => (
        <Link key={it.href} href={it.href} className="flex flex-1 flex-col items-center py-2 text-xs">
          <span aria-hidden={it.icon}</span>
          <span>{it.label}</span>
        </Link>
      ))}
    </nav>
  )
}

```

```

    )
  }

  export default function RootLayout({ children }: { children: React.ReactNode }) {
    return (
      <html lang="id" suppressHydrationWarning>
        <body className="min-h-dvh bg-white pb-16 text-neutral-900 antialiased dark:bg-neutral-950 dark:text-neutral-100">
          <Providers>{children}</Providers>
          <BottomNav />
        </body>
      </html>
    )
  }

```

7.13 globals.css + Tailwind dark mode

```

/* src/app/globals.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

:root { color-scheme: light dark; }
html.dark { color-scheme: dark; }

```

```

// tailwind.config.ts
import type { Config } from "tailwindcss"
export default {
  darkMode: "class", // di-toggle via class "dark" pada <html>
  content: ["./src/**/*.{ts,tsx}"],
  theme: { extend: {} },
  plugins: [],
} satisfies Config

```



```
// src/lib/theme.ts – helper dark mode (disimpan ke Dexie s
ettings)
import { db } from "../db/db"
export async function applyTheme(mode: "light" | "dark") {
  document.documentElement.classList.toggle("dark", mode
=== "dark")
  await db.settings.put({ key: "theme", value: mode })
}
export async function initTheme() {
  const saved = await db.settings.get("theme")
  const mode = (saved?.value as "light" | "dark") ??
(window.matchMedia("(prefers-color-scheme: dark)").
matches ? "dark" : "light")
  document.documentElement.classList.toggle("dark", mode
=== "dark")
  return mode
}
```

7.14 ConnectionBadge.tsx

```
// src/components/ConnectionBadge.tsx
"use client"
import { useEffect, useState } from "react"

export function ConnectionBadge() {
  const [online, setOnline] = useState(true)
  useEffect(() => {
    const on = () => setOnline(true)
    const off = () => setOnline(false)
    setOnline(navigator.onLine)
    window.addEventListener("online", on)
    window.addEventListener("offline", off)
    return () => { window.removeEventListener("online",
on); window.removeEventListener("offline", off) }
  }, [])
  return (
    <span
```

```

        role="status"
        className={`inline-flex items-center gap-1 rounded-full px-2 py-0.5 text-xs ${online ? "bg-green-500/15 text-green-600" : "bg-neutral-500/15 text-neutral-500"}`}
      >
        <span className={`h-2 w-2 rounded-full ${online ? "bg-green-500" : "bg-neutral-400"}`} />
        {online ? "online" : "offline"}
      </span>
    )
  }
}

```

7.15 RiskCalculator.tsx

```

// src/components/RiskCalculator.tsx
"use client"
import { useMemo, useState } from "react"
import { calcPosition } from "@lib/risk"

export function RiskCalculator({ defaultEntry = 0, lotSize = 100 }: { defaultEntry?: number; lotSize?: number }) {
  const [capital, setCapital] = useState(10_000_000)
  const [riskPct, setRiskPct] = useState(1)
  const [entry, setEntry] = useState(defaultEntry)
  const [stopLoss, setStopLoss] = useState(0)

  const result = useMemo(() => {
    try { return { ok: true as const, ...calcPosition({ capital, riskPct, entry, stopLoss, lotSize }) } }
    catch (e: any) { return { ok: false as const, error: e.message as string } }
  }, [capital, riskPct, entry, stopLoss, lotSize])

  const num = (v: number, set: (n: number) => void, label: string) => (
    <label className="flex flex-col text-sm">
      <span className="opacity-70">{label}</span>
      <input type="number" inputMode="decimal" value=

```

```

{v}
        onChange={e) => set(Number(e.target.valu
e))}
        className="rounded-lg border p-2 dark:bg-ne
utral-900" />
      </label>
    )

    return (
      <div className="space-y-3 rounded-xl border p-3 dar
k:border-neutral-800">
        <div className="grid grid-cols-2 gap-2">
          {num(capital, setCapital, "Modal (Rp))"}
          {num(riskPct, setRiskPct, "Risk %")}
          {num(entry, setEntry, "Entry")}
          {num(stopLoss, setStopLoss, "Stop Loss")}
        </div>
        {result.ok ? (
          <dl className="grid grid-cols-2 gap-2 text-
sm">
            <div><dt className="opacity-60">Risk no
minal</dt><dd className="font-semibold">Rp {Math.round(resu
lt.riskNominal).toLocaleString("id-ID")}</dd></div>
            <div><dt className="opacity-60">Risk /
lembar</dt><dd className="font-semibold">Rp {result.riskPer
Share.toLocaleString("id-ID")}</dd></div>
            <div><dt className="opacity-60">Lembar
</dt><dd className="font-semibold">{result.shares.toLocaleS
tring("id-ID")}</dd></div>
            <div><dt className="opacity-60">Lot</dt
><dd className="font-semibold">{result.lots.toLocaleString
("id-ID")}</dd></div>
          </dl>
        ) : (
          <p className="text-sm text-red-500">{resul
t.error}</p>
        )}
      </div>
    )
  }
}

```

```
)  
}
```

7.16 TradingPlanForm.tsx

```
// src/components/TradingPlanForm.tsx  
"use client"  
import { useEffect, useState } from "react"  
import { useLiveQuery } from "dexie-react-hooks"  
import { db, type TradingPlan } from "@lib/db/db"  
import { validateLongPlan } from "@lib/plan"  
  
const empty = { entryLow: 0, entryHigh: 0, stopLoss: 0, tp  
1: 0, tp2: 0, riskPct: 1, setup: "", thesis: "" }  
  
export function TradingPlanForm({ code }: { code: string })  
{  
  const existing = useLiveQuery(() => db.plans.where("cod  
e").equals(code).first(), [code])  
  const [form, setForm] = useState<Omit<TradingPlan, "id"  
| "code" | "direction" | "updatedAt">>>(empty)  
  const [msg, setMsg] = useState("")  
  
  useEffect(() => { if (existing) setForm({ ...empty, ...  
existing }) }, [existing])  
  
  const set = (k: keyof typeof form) => (v: string) =>  
    setForm((f) => ({ ...f, [k]: k === "setup" || k ===  
"thesis" ? v : Number(v) })))  
  
  async function save() {  
    setMsg("")  
    const check = validateLongPlan(form)  
    if (!check.ok) { setMsg(check.error!); return }  
    const payload: TradingPlan = { ...form, code, direc  
tion: "long", updatedAt: Date.now() }  
    if (existing?.id) await db.plans.update(existing.i  
d, payload)
```

```

        else await db.plans.add(payload)
        setMsg("Tersimpan ✓")
    }

    const field = (k: keyof typeof form, label: string, type: "number" | "text" = "number") => (
        <label className="flex flex-col text-sm">
            <span className="opacity-70">{label}</span>
            <input type={type} inputMode={type === "number"
? "decimal" : undefined}
                value={form[k] as any} onChange={(e) => set
(k)(e.target.value)}
                className="rounded-lg border p-2 dark:bg-ne
utral-900" />
        </label>
    )

    return (
        <div className="space-y-3 rounded-xl border p-3 dar
k:border-neutral-800">
            <div className="grid grid-cols-2 gap-2">
                {field("entryLow", "Entry Low")}
                {field("entryHigh", "Entry High")}
                {field("stopLoss", "Stop Loss")}
                {field("riskPct", "Risk %")}
                {field("tp1", "TP1")}
                {field("tp2", "TP2")}
            </div>
            {field("setup", "Setup", "text")}
            {field("thesis", "Thesis / catatan", "text")}
            <button onClick={save} className="w-full rounde
d-lg bg-blue-600 py-2 text-white">Simpan Plan</button>
            {msg && <p className={`text-sm ${msg.includes
("✓")} ? "text-green-600" : "text-red-500"}`>{msg}</p>}
        </div>
    )
}

```

7.17 RsiPanel.tsx

```
// src/components/RsiPanel.tsx
"use client"
import { useEffect, useRef } from "react"
import { createChart, type IChartApi } from "lightweight-charts"
import { rsi } from "@lib/indicators"

export function RsiPanel({ candles }: { candles: { time: string; close: number }[] }) {
  const ref = useRef<HTMLDivElement>(null)
  const chart = useRef<IChartApi | null>(null)
  useEffect(() => {
    if (!ref.current) return
    chart.current = createChart(ref.current, { height: 120, layout: { background: { color: "transparent" }, textColor: "#888" } })
    const series = chart.current.addLineSeries({ lineWidth: 1, color: "#a855f7" })
    const values = rsi(candles.map((c) => c.close), 14)
    series.setData(values.map((v, i) => (v == null ? null : { time: candles[i].time, value: v }))).filter(Boolean) as any
    for (const lvl of [70, 30]) series.createPriceLine({ price: lvl, color: "#888", lineStyle: 2, lineWidth: 1, axisLabelVisible: true, title: String(lvl) })
    const ro = new ResizeObserver(() => chart.current?.applyOptions({ width: ref.current!.clientWidth }))
    ro.observe(ref.current)
    return () => { ro.disconnect(); chart.current?.remove() }
  }, [candles])
  return <div ref={ref} className="w-full" aria-label="RSI 14" />
}
```

7.18 /symbol/[code]/page.tsx (detail)

```

// src/app/symbol/[code]/page.tsx
"use client"
import { use, useState } from "react"
import { useQuote, useDaily } from "@/hooks/useMarket"
import { PriceChart } from "@/components/PriceChart"
import { RsiPanel } from "@/components/RsiPanel"
import { TradingPlanForm } from "@/components/TradingPlanForm"
import { RiskCalculator } from "@/components/RiskCalculator"
import { db } from "@/lib/db/db"

export default function SymbolPage({ params }: { params: Promise<{ code: string }> }) {
  const { code } = use(params)
  const quote = useQuote(code)
  const daily = useDaily(code)
  const [savedMsg, setSavedMsg] = useState("")

  async function addJournalEntry() {
    await db.journal.add({
      code,
      date: new Date().toISOString().slice(0, 10),
      entry: quote.data?.price ?? 0,
      exit: 0, qtyLot: 0, createdAt: Date.now(),
    })
    setSavedMsg("Entry journal dibuat – lengkapi di halaman Journal")
  }

  const candles = daily.data?.candles ?? []
  const up = (quote.data?.changePercent ?? 0) >= 0

  return (
    <main className="mx-auto max-w-md space-y-4 p-4">
      <header>
        <h1 className="text-2xl font-bold">{code}</h1>

```

```

        <p className={up ? "text-green-500" : "text
-red-500"}>
            {quote.isError ? "data gagal / limit" :
`${quote.data?.price ?? "-"} (${quote.data?.changePercent?.
toFixed(2) ?? "-"}%)`}
        </p>
    </header>

    <section className="space-y-2">
        <h2 className="text-sm font-semibold opacit
y-70">Chart 1D – SMA(20) · EMA(20)</h2>
        {daily.isLoading ? <p className="text-sm op
acity-60">memuat chart...</p>
            : daily.isError ? <p className="text-sm
text-red-500">Gagal memuat data (rate limit / no data). <bu
tton className="underline" onClick={() => daily.refetch()}>
Coba lagi</button></p>
            : candles.length === 0 ? <p className
="text-sm opacity-60">Tidak ada data.</p>
            : (<><PriceChart candles={candles} /><R
siPanel candles={candles} /></>)}
    </section>

    <button onClick={addJournalEntry} className="w-
full rounded-lg bg-emerald-600 py-2 text-white">+ Add journ
al entry</button>
    {savedMsg && <p className="text-sm text-emerald
-600">{savedMsg}</p>}

    <details>
        <summary className="cursor-pointer font-sem
ibold">Trading Plan</summary>
        <div className="mt-2"><TradingPlanForm code
={code} /></div>
    </details>

    <details>
        <summary className="cursor-pointer font-sem

```



```

ibold">Risk Calculator</summary>
      <div className="mt-2"><RiskCalculator defau
ltEntry={quote.data?.price ?? 0} /></div>
      </details>
    </main>
  )
}

```

7.19 /journal/page.tsx

```

// src/app/journal/page.tsx
"use client"
import { useLiveQuery } from "dexie-react-hooks"
import { db } from "@/lib/db/db"
import { journalStats } from "@/lib/stats"

export default function JournalPage() {
  const entries = useLiveQuery(() => db.journal.orderBy
("date").reverse().toArray(), [], [])
  const stats = journalStats(entries ?? [])

  const Stat = ({ label, value }: { label: string; value:
string }) => (
    

{label}



{value}


```

```

        <Stat label="Winrate" value={`$${(stats.winrate * 100).toFixed(0)}%`} />
        <Stat label="Avg R" value={stats.avgR.toFixed(2)} />
        <Stat label="Expectancy" value={stats.expectancy.toFixed(2)} />
      </div>
      <ul className="space-y-2">
        {(entries ?? []).map((e) => (
          <li key={e.id} className="flex items-center justify-between rounded-xl border p-3 dark:border-neutral-800">
            <div><span className="font-semibold">{e.code}</span> <span className="text-xs opacity-60">{e.date}</span></div>
            <div className={`font-bold ${e.rMultiple ?? 0} >= 0 ? "text-green-500" : "text-red-500"}`>
              {(e.rMultiple ?? 0).toFixed(2)} R</div>
            </li>
          )))
      </ul>
    </main>
  )
}

```

7.20 /settings/page.tsx (tema, notifikasi, lotSize, provider status, export/import)

```

// src/app/settings/page.tsx
"use client"
import { useEffect, useState } from "react"
import { applyTheme, initTheme } from "@lib/theme"
import { ensureNotifyPermission } from "@lib/alerts"
import { exportData, importData } from "@lib/backup"
import { db } from "@lib/db/db"

export default function SettingsPage() {
  const [theme, setTheme] = useState<"light" | "dark">("l

```

```

ight")
    const [lotSize, setLotSize] = useState(100)
    const [notif, setNotif] = useState<NotificationPermissi
on>("default")
    const [msg, setMsg] = useState("")

    useEffect(() => {
        initTheme().then(setTheme)
        db.settings.get("lotSize").then((s) => s && setLotS
ize(Number(s.value)))
        if ("Notification" in window) setNotif(Notification
n.permission)
    }, [])

    async function toggleTheme() {
        const next = theme === "dark" ? "light" : "dark"
        await applyTheme(next); setTheme(next)
    }

    async function saveLot(v: number) { setLotSize(v); awai
t db.settings.put({ key: "lotSize", value: v }) }
    async function reqNotif() { await ensureNotifyPermissio
n(); setNotif(Notification.permission) }

    async function doExport() {
        const json = await exportData()
        const blob = new Blob([json], { type: "application/
json" })
        const a = document.createElement("a")
        a.href = URL.createObjectURL(blob)
        a.download = `idx-helper-backup-${new Date().toISOSt
ring().slice(0, 10)}.json`
        a.click(); URL.revokeObjectURL(a.href)
    }

    async function doImport(file: File) {
        setMsg("")
        try { await importData(await file.text()); setMsg
("Import berhasil ✓") }
        catch { setMsg("File tidak valid / schema salah") }
    }

```

```

    }

    return (
      <main className="mx-auto max-w-md space-y-4 p-4">
        <h1 className="text-xl font-bold">Settings</h1>

        <div className="flex items-center justify-between rounded-xl border p-3 dark:border-neutral-800">
          <span>Dark mode</span>
          <button onClick={toggleTheme} className="rounded-lg border px-3 py-1">{theme === "dark" ? "On" : "Off"}</button>
        </div>

        <label className="flex items-center justify-between rounded-xl border p-3 dark:border-neutral-800">
          <span>Lot size (lembar/lot)</span>
          <input type="number" value={lotSize} onChange={(e) => saveLot(Number(e.target.value))} className="w-24 rounded-lg border p-1 text-right dark:bg-neutral-900" />
        </label>

        <div className="flex items-center justify-between rounded-xl border p-3 dark:border-neutral-800">
          <span>Notifikasi: {notif}</span>
          <button onClick={reqNotif} className="rounded-lg border px-3 py-1">Minta izin</button>
        </div>

        <div className="flex gap-2">
          <button onClick={doExport} className="flex-1 rounded-lg bg-blue-600 py-2 text-white">Export JSON</button>

          <label className="flex-1 cursor-pointer rounded-lg border py-2 text-center">
            Import JSON
            <input type="file" accept="application/json" className="hidden"

```

```

                                onChange={(e) => e.target.files?.
[0] && doImport(e.target.files[0])} />
                                </label>
                            </div>
                            {msg && <p className="text-sm">{msg}</p>}
                            <p className="text-xs opacity-60">Export tidak
menyertakan API key (keys hanya ada di server).</p>
                        </main>
                    )
                }

```

7.21 /api/search/route.ts

```

// src/app/api/search/route.ts
import { NextResponse } from "next/server"
import { market } from "@lib/market"
import { RateLimitError } from "@lib/market/provider"

export const runtime = "nodejs"

export async function GET(req: Request) {
    const q = new URL(req.url).searchParams.get("q")?.trim
    ()
    if (!q) return NextResponse.json({ error: "q required"
}, { status: 400 })
    try {
        const { data, provider } = await market.searchSymbo
l(q)
        return NextResponse.json({ results: data, provider
}, {
            headers: { "Cache-Control": "s-maxage=86400, st
ale-while-revalidate=604800" },
        })
    } catch (e) {
        if (e instanceof RateLimitError) return NextRespons
e.json({ error: "rate_limit" }, { status: 429 })
        return NextResponse.json({ error: "upstream" }, { s
tatus: 502 })
    }
}

```

```
}  
}
```

7.22 package.json (scripts) + vitest.config.ts

```
{  
  "name": "idx-trading-helper",  
  "private": true,  
  "scripts": {  
    "dev": "next dev",  
    "build": "next build",  
    "start": "next start",  
    "lint": "next lint",  
    "test": "vitest run",  
    "test:watch": "vitest"  
  },  
  "dependencies": {  
    "next": "^14.2.0",  
    "react": "^18.3.0",  
    "react-dom": "^18.3.0",  
    "dexie": "^4.0.0",  
    "dexie-react-hooks": "^1.1.7",  
    "@tanstack/react-query": "^5.0.0",  
    "lightweight-charts": "^4.2.0",  
    "zod": "^3.23.0"  
  },  
  "devDependencies": {  
    "next-pwa": "^5.6.0",  
    "vitest": "^2.0.0",  
    "@vitest/coverage-v8": "^2.0.0",  
    "typescript": "^5.4.0",  
    "tailwindcss": "^3.4.0",  
    "autoprefixer": "^10.4.0",  
    "postcss": "^8.4.0"  
  }  
}
```

```
// vitest.config.ts
import { defineConfig } from "vitest/config"
export default defineConfig({
  test: { environment: "node", include: ["src/**/*.test.ts"] },
})
```

7.23 Test tambahan (risk / plan / stats)

```
// src/lib/risk.test.ts
import { describe, it, expect } from "vitest"
import { calcPosition } from "./risk"

describe("calcPosition", () => {
  it("hitung shares & lot", () => {
    const r = calcPosition({ capital: 10_000_000, riskPct: 1, entry: 1000, stopLoss: 950, lotSize: 100 })
    expect(r.riskNominal).toBe(100_000)
    expect(r.riskPerShare).toBe(50)
    expect(r.shares).toBe(2000)
    expect(r.lots).toBe(20)
  })
  it("tolak SL = entry", () => {
    expect(() => calcPosition({ capital: 1, riskPct: 1, entry: 100, stopLoss: 100 })).toThrow()
  })
  it("tolak modal/risk <= 0", () => {
    expect(() => calcPosition({ capital: 0, riskPct: 1, entry: 100, stopLoss: 90 })).toThrow()
  })
})
```

```
// src/lib/plan.test.ts
import { describe, it, expect } from "vitest"
import { validateLongPlan } from "./plan"

describe("validateLongPlan", () => {
```

```

    it("valid: SL < EL <= EH < TP1 <= TP2", () => {
        expect(validateLongPlan({ stopLoss: 90, entryLow: 100, entryHigh: 105, tp1: 110, tp2: 120 })).ok().toBe(true)
    })
    it("tolak SL >= EntryLow", () => {
        const r = validateLongPlan({ stopLoss: 100, entryLow: 100, entryHigh: 105, tp1: 110, tp2: 120 })
        expect(r.ok).toBe(false)
    })
    it("tolak EntryHigh >= TP1", () => {
        expect(validateLongPlan({ stopLoss: 90, entryLow: 100, entryHigh: 110, tp1: 110, tp2: 120 })).ok().toBe(false)
    })
})

```

```

// src/lib/stats.test.ts
import { describe, it, expect } from "vitest"
import { journalStats } from "./stats"

describe("journalStats", () => {
    it("0 trade", () => {
        expect(journalStats([])).toEqual({ trades: 0, winrate: 0, avgR: 0, expectancy: 0 })
    })
    it("winrate & expectancy", () => {
        const s = journalStats([
            { code: "A", date: "2026-01-01", entry: 1, exit: 2, qtyLot: 1, rMultiple: 2, createdAt: 0 },
            { code: "A", date: "2026-01-02", entry: 1, exit: 0.5, qtyLot: 1, rMultiple: -1, createdAt: 0 },
        ])
        expect(s.trades).toBe(2)
        expect(s.winrate).toBe(0.5)
        expect(s.avgR).toBeCloseTo(0.5)
        expect(s.expectancy).toBeCloseTo(0.5) // 0.5*2 - 0.5*1
    })
})

```



```
} )  
})
```

7.24 Wiring akhir & catatan

- Tambahkan `dexie-react-hooks` (sudah ada di `package.json`).
- `page.tsx` Dashboard sudah memakai `Providers` via `layout.tsx` — tidak perlu wrap ulang.
- Untuk evaluasi alert otomatis, panggil `evaluateAlerts(code, price, rsiNow)` di dalam `onSuccess` query quote pada halaman detail/dashboard.
- Halaman `journal` MVP menampilkan list + statistik; form edit detail entry (exit, qty, fees, emosi, R) bisa ditambahkan sebagai modal pada iterasi M4.

8. Checklist Testing

Unit (vitest)

- ☐ `normalizeUserCode` : uppercase, trim, strip `.JK/.JKT/:IDX`, reject invalid.
- ☐ `toProviderSymbol` : twelvedata=base, alphavantage= `.JKT`.
- ☐ `sma/ema/rsi` : nilai benar + handle data < period (null).
- ☐ `calcPosition` : shares/lot benar; throw saat SL=entry & input ≤ 0.
- ☐ `validateLongPlan` : semua aturan urutan + pesan error spesifik.
- ☐ `journalStats` : winrate/avgR/expectancy; handle 0 trade.
- ☐ `backupSchema` : tolak JSON invalid; pastikan tidak ada field apiKey.

Manual QA Mobile

- ☐ Install PWA dari Chrome/Safari mobile (banner/Add to Home Screen).
- ☐ Offline: watchlist/journal/plan tetap terbaca; market data tampilkan stale + badge offline.
- ☐ Rate limit (429): UI tampil pesan limit + retry, fallback ke Alpha Vantage berfungsi.
- ☐ Invalid symbol (mis. `XXXX`): pesan "symbol tidak ditemukan".

- ☐ Dark mode & kontras (WCAG AA), target tap $\geq 44\text{px}$, navigasi keyboard/screen reader pada form.
- ☐ Notifikasi: izin granted $\rightarrow$ notif; denied $\rightarrow$ toast fallback.
- ☐ Export $\rightarrow$ Import roundtrip mengembalikan data identik.

9. Disclaimer & Batasan Data



Edukasi, bukan nasihat keuangan. Data dapat terlambat/berbeda.

- Aplikasi tidak mengeksekusi order dan bukan auto-trading.
- Data market berasal dari provider pihak ketiga (Twelve Data / Alpha Vantage) pada free tier: kemungkinan **delay (EOD/15 menit)**, **rate limit**, dan **perbedaan harga** vs platform broker.
- API key hanya disimpan di environment variable server, tidak pernah dikirim ke client maupun ikut di-export.
- Keputusan trading sepenuhnya tanggung jawab pengguna.

Menjalankan

```
npm install
cp .env.local.example .env.local    # isi TWELVE_DATA_API_KEY & ALPHA_VANTAGE_API_KEY
npm run dev                          # http://localhost:3000
0
npm run test                         # unit (vitest)
```

Untuk uji PWA installable, build production: `npm run build && npm start`, lalu buka dari browser mobile dan pilih **Add to Home Screen**.